



中山大學
SUN YAT-SEN UNIVERSITY

本科生实验报告

实验课程：操作系统原理实验

任课教师：刘宁

实验题目：内存管理

专业名称：计算机科学与技术（人工智能与大数据方向）

学生姓名：陶昕璨

学生学号：24325242

实验地点：实验中心 D501

实验时间：2026/5/9-2026/5/13

目录

一、实验概述.....	1
二、预备知识与实验环境.....	1
三、实验任务.....	1
四、实验步骤与实验结果.....	1
实验任务一.....	1
实验任务二.....	8
实验任务三.....	14
实验任务四.....	23
五、实验总结与心得体会.....	26
六、附录：参考资料清单.....	27

Section 1 实验概述

在本次实验中，首先学习如何使用**位图和地址池来管理资源**。然后，我们将实现在**物理地址空间下的内存管理**。接着，我们将会学习并开启**二级分页机制**。在开启分页机制后，我们将实现在**虚拟地址空间下的内存管理**

Section 2 预备知识与实验环境

- 预备知识：x86 汇编语言程序设计、Linux 系统命令行工具、内核/Shell/GDB 基础、C++/C 程序运行
- 实验环境：
 - 虚拟机版本/处理器型号：openSUSE (64-bit)
 - 代码编辑环境：Vim (终端) / vscode
 - 代码编译工具：GCC、g++、nasm、ld (链接器)、qemu-system-i386、GDB
 - 重要三方库信息：libc6-dev-i386、gcc-multilib

Section 3 实验任务

- 实验任务 1：物理页内存管理的实现
- 实验任务 2：二级分页机制的实现
- 实验任务 3：虚拟页内存管理的实现
- 实验任务 4：页面置换算法

Section 4 实验步骤与实验结果

----- 实验任务 1 -----

- 任务要求：物理页内存管理的实现——复现“物理页内存管理”代码
- 结合代码分析位图，地址池，物理页管理的初始化过程，以及物理页进行分配和释放的实现思路。

构造测试用例来分析物理页内存管理的实现是否存在 bug。如果存在，则尝试修复并再次测试。否则，结合测试用例简要分析物理页内存管理的实现的正确性。

- 思路分析：理解代码，构造案例验证 1. 分配释放资源正确 2. 边界正确 3. 池间隔离
- 实验步骤：

①位图初始化

▲BitMap 类的封装：

```
class BitMap
{
public:
    // 被管理的资源个数，bitmap的总位数
    int length;
    // bitmap的起始地址
    char *bitmap;
public:
    BitMap();
    void initialize(char *bitmap, const int length);
    // 获取第index个资源的状态，true=allocated, false=free
    bool get(const int index) const;
    // 设置第index个资源的状态，true=allocated, false=free
    void set(const int index, const bool status);
    // 分配count个连续的资源，若没有则返回-1，否则返回分配的第1个资源单元序号
    int allocate(const int count);
    // 释放第index个资源开始的count个资源
    void release(const int index, const int count);
    char *getBitmap();
    int size() const;
private:
    // 禁止Bitmap之间的赋值
    BitMap(const BitMap &) {}
    void operator=(const BitMap&) {}
};
```

▲初始化函数方法：赋初值

```

BitMap::BitMap()
{
}

void BitMap::initialize(char *bitmap, const int length)
{
    //初始赋值
    this->bitmap = bitmap;
    this->length = length;

    int bytes = ceil(length, 8); //向上取整，得到length位转换成字节最少所需字节数

    //初始所有资源都未使用
    for (int i = 0; i < bytes; ++i)
    {
        bitmap[i] = 0;
    }
}

```

②地址池的初始化

▲AddressPool 类的封装

```

class AddressPool
{
public:
    BitMap resources; //对地址空间进行管理，标识地址池中哪些页是空闲的，哪些页是被分配的
    int startAddress; //起始地址

public:
    AddressPool();
    // 初始化地址池
    void initialize(char *bitmap, const int length, const int startAddress);
    // 从地址池中分配count个连续页，成功则返回第一个页的地址，失败则返回-1
    int allocate(const int count);
    // 释放若干页的空间
    void release(const int address, const int amount);
};

```

▲初始化函数方法：对 startAddress 赋初始值，而 resources 则调用位图的初始化函数方法

```

// 设置地址池BitMap
void AddressPool::initialize(char *bitmap, const int length, const int startAddress)
{
    resources.initialize(bitmap, length);
    this->startAddress = startAddress;
}

```

③物理页管理的初始化

▲物理页管理类的封装：

```

//防止造成不同空间的冲突
enum AddressPoolType
{
    USER,
    KERNEL
};

class MemoryManager
{
public:
    int totalMemory;
    // 内核物理地址池
    AddressPool kernelPhysical;
    // 用户物理地址池
    AddressPool userPhysical;

public:
    MemoryManager();

    // 初始化地址池
    void initialize();

    // 从type类型的物理地址池中分配count个连续的页
    // 成功，返回起始地址；失败，返回0
    int allocatePhysicalPages(enum AddressPoolType type, const int count);

    // 释放从paddr开始的count个物理页
    void releasePhysicalPages(enum AddressPoolType type, const int startAddress, const int count);

    int getTotalMemory();
};

```

▲初始化函数方法：totalMemory 通过实模式中断获得内存大小，而内核物理地址池子和用户地址池通过调用地址池的初始化函数来初始化

```

void MemoryManager::initialize()
{
    this->totalMemory = 0;
    //调用读取实模式下使用中断获取的内存大小的函数
    this->totalMemory = getTotalMemory();

    // 预留的内存，1MB以上的剩余部分存放内核页表
    int usedMemory = 256 * PAGE_SIZE + 0x100000;
    if(this->totalMemory < usedMemory) {
        printf("memory is too small, halt.\n");
        asm_halt();
    }

    // 剩余的空闲的内存
    int freeMemory = this->totalMemory - usedMemory;

    int freePages = freeMemory / PAGE_SIZE;
    //等分内核物理地址空间和用户物理地址空间
    int kernelPages = freePages / 2;
    int userPages = freePages - kernelPages;

    //计算起始地址，用户物理地址空间紧跟在内核物理地址空间后
    int kernelPhysicalStartAddress = usedMemory;
    int userPhysicalStartAddress = usedMemory + kernelPages * PAGE_SIZE;

    int kernelPhysicalBitMapStart = BITMAP_START_ADDRESS;
    int userPhysicalBitMapStart = kernelPhysicalBitMapStart + ceil(kernelPages, 8);

    //对两部分空间的地支持初始化
    kernelPhysical.initialize((char *)kernelPhysicalBitMapStart, kernelPages, kernelPhysicalStartAddress);
    userPhysical.initialize((char *)userPhysicalBitMapStart, userPages, userPhysicalStartAddress);

    //打印基本信息
}

```

其中 `getTotalMemory()`：注意要先在实模式下调用中断获取内存大小，然后将内存大小写入一个固定的地址

```
int MemoryManager::getTotalMemory()
{
    if(!this->totalMemory)
    {
        int memory = *((int *)MEMORY_SIZE_ADDRESS);
        // ax寄存器保存的内容
        int low = memory & 0xffff;
        // bx寄存器保存的内容
        int high = (memory >> 16) & 0xffff;

        this->totalMemory = low * 1024 + high * 64 * 1024;
    }

    return this->totalMemory;
}
```

④物理页的分配（分配和释放几乎是类似而相反的操作，所以以下以分析分配的代码为例）

▲物理页管理的分配：调用地址池的分配函数方法

```
int MemoryManager::allocatePhysicalPages(enum AddressPoolType type, const int count)
{
    int start = -1;

    if (type == AddressPoolType::KERNEL)
    {
        start = kernelPhysical.allocate(count);
    }
    else if (type == AddressPoolType::USER)
    {
        start = userPhysical.allocate(count);
    }

    //若不存在返回0，若有满足分配则start
    return (start == -1) ? 0 : start;
}
```

▲地址池的分配：调用位图的分配函数方法

```
// 从地址池中分配count个连续页
int AddressPool::allocate(const int count)
{
    int start = resources.allocate(count);
    return (start == -1) ? -1 : (start * PAGE_SIZE + startAddress);
}
```

▲位图的分配：

```

int BitMap::allocate(const int count)
{
    if (count == 0)
        return -1;

    int index, empty, start;

    index = 0;
    while (index < length)
    {
        // 越过已经分配的资源
        while (index < length && get(index))
            ++index;

        // 不存在连续的count个资源
        if (index == length)
            return -1;

        // 找到1个未分配的资源
        // 检查是否存在从index开始的连续count个资源
        empty = 0;
        start = index;
        while ((index < length) && (!get(index)) && (empty < count))
        {
            ++empty;
            ++index;
        }

        // 存在连续的count个资源
        if (empty == count)
        {
            for (int i = 0; i < count; ++i)
            {
                //位图置1表示已分配地址资源
                set(start + i, true);
            }

            return start;
        }
    }

    return -1;
}

```

● 实验结果展示：

1. 内存池初始化分析

- 总容量： 126 MB
- 地址分配：
 - 内核池：起始地址为 0x200000 。
 - 用户池：起始地址为 0x4070000。

- 计算验证：内核池有 15984 个页（约 62MB）。 $0x200000 + 15984 \times 4096 = 0x200000 + 0x3E70000 = 0x4070000$ 。这证明了用户池紧跟在内核池之后。
 - 位图（BitMap）位置：内核位图始于 0x10000，用户位图始于 0x107CE。两者互不重叠。
-

2. 测试用例行为分析

Test 1 & 2: 常规分配与连续性

- 现象：第一次分配 3 页得到 0x200000，第二次分配 5 页得到 0x203000。
- 分析： $0x203000 - 0x200000 = 0x3000$ ，正好是 3 个页的大小（ $3 \times 4096 = 12288$ 字节）。

Test 3 & 4: 释放与碎片重用

- 现象：释放了 0x200000 开始的 3 个页后，重新请求 2 个页，返回地址依然是 0x200000。
- 分析：release 函数成功将位图的前几位清零（Reset），并且 allocate 的扫描算法能够优先填补内存低地址处的“空洞”
- 结论：内存释放与回收逻辑正确，没有出现内存泄漏

Test 5: 边界异常处理

- 现象：Overflow allocation failed as expected。
- 分析：当请求超过物理池剩余容量的页数时，系统返回了 0（或相应错误标识），没有发生数组越界或非法分配。
- 结论：代码具备基本的健壮性。

Test 6: 池间隔离性

- 现象：从用户池分配 1 页，地址为 0x4070000。
 - 分析：该地址准确指向了初始化时定义的 user_pool_start_address。
 - 结论：内核与用户物理空间实现了逻辑隔离。
-

3. 实验总结

通过本次测试，可以确认物理页内存管理模块（BitMap + AddressPool +

MemoryManager) 已经具备了以下能力:

1. 物理地址计算
2. 可靠的资源状态跟踪
3. 有效的内存回收与再利用机制

```
total memory: 133038080 bytes ( 126 MB )
kernel pool
  start address: 0x200000
  total pages: 15984 ( 62 MB )
  bitmap start address: 0x10000
user pool
  start address: 0x4070000
  total pages: 15984 ( 62 MB )
  bit map start address: 0x107CE
--- Starting Memory Manager Test ---
Test 1: Allocate 3 pages, addr: 0x200000
Test 2: Allocate 5 pages, addr: 0x203000
Test 3: Release 3 pages at 0x200000
Test 4: Re-allocate 2 pages, addr: 0x200000 (Expected: 0x200000)
Test 5: Overflow allocation failed as expected.
Test 6: User pool allocate 1 page, addr: 0x4070000
--- Memory Manager Test Completed ---
```

----- 实验任务 2 -----

- 任务要求: 二级分页机制的实现——复现“二级分页机制”代码
- . 实现内存的申请和释放, 保存实验截图并对能够在虚拟地址空间中进行内存管理, 截图并给出过程解释(比如: 说明哪些输出信息描述虚拟地址, 哪些输出信息描述物理地址)。注意: 建议使用的物理地址或虚拟地址信息与学号相关联(比如学号后四位作为页内偏移), 作为报告独立完成的个人信息表征。
- . 相比于一级页表, 二级页表的开销是增大的, 但操作系统中往往使用的是二级页表而不是一级页表。结合实验过程, 说说相比于一级页表, 使用二级页表会带来哪些优势。
- 思路分析:
 1. 内存空间的规划与布局
 - 选址对齐: 页目录表和页表必须放置在物理地址为 4KB 整数倍的位置

（低 12 位为 0），这是硬件 MMU 寻址的硬性要求。

- **预留空间：**实验将页目录表设在 0x100000（1MB）处，紧随其后分配页表空间。这种布局规避了 MBR 和 Bootloader 所在的低端区域，确保了管理结构的安全性。

2. 构建二级映射层级结构

- **物理页表初始化：**
 - **平滑过渡（Identity Mapping）：**将虚拟地址 0~1MB 映射到物理地址 0~1MB。保证开启分页的一瞬间，CPU 执行的下一条指令依然在预期的物理位置。
 - **权限设定：**为页表项设置属性（如 0x7），代表该页存在（P=1）、可读写（RW=1）且用户可访问（US=1）。
- **页目录表（PDE）初始化：**
 - **低端映射：**directory[0] 指向上述页表，负责处理低端 4MB 空间的访问。
 - **内核高端映射：**directory[768] 同样指向该页表。这利用了二级分页的灵活性，将虚拟地址 3GB（768 \times 4MB）处开始的内核空间也映射到物理地址 0~1MB。
 - **自映射技巧：**设置 directory[1023] 指向页目录表自身，为后续在虚拟地址空间下动态修改页表提供了便捷入口

3. 激活硬件 MMU 转换开关

配置好内存数据后，需要通知 CPU 硬件切换工作模式。

- **加载 CR3 寄存器：**将页目录表的物理基地址写入 cr3。此时，CPU 的地址转换部件（MMU）已经锁定了“字典”的位置。
- **开启 PG 位：**通过汇编代码修改 cr0 寄存器的第 31 位（PG 位）。
 - **转折点：**一旦该位置 1，CPU 发出的所有地址都会被视为虚拟地址，自动触发“页目录 -> 页表 -> 物理地址”的转换链路。

4. 虚拟地址空间管理与联动

- **虚拟地址池分配：**管理连续的虚拟路径（线性地址空间）。
- **物理地址池分配：**管理零散的物理落脚点（物理页）。

- 动态挂载：每当申请内存时，通过修改页表将虚拟地址与物理地址关联

5. 二级分页机制：

本实验在 32 位实模式下进行，具有 32 位地址空间，设计的二级分页机制为，1 个页目录具有 1024 个页表项（大小为 4B），每一个页表项有 1024 个物理页（大小也为 4B），每一个物理页大小为 4KB。

● 实验步骤：

①规划好页目录表和页表在内存中的位置，然后初始化

▲页目录表和页表是需要在内存中特意地划分出位置来存放的。所以，我们需要在内存中指定页目录表和页表存放的位置。

▲页目录表和页表的物理地址必须是 4KB 的整数倍，也就是低 12 位为 0

▲规定了页目录表的位置后，我们根据线性地址空间的大小来确定需要分配的页表的数量和位置，向页目录表中写入页表对应的页目录项

31	12	11	9	8	7	6	5	4	3	2	1	0
页表物理地址 31~12		AVL	G	PAT	D	A	PCD	PWT	US	RW	P	

▲写完页目录项后便可以在页表中写页表项

31	12	11	9	8	7	6	5	4	3	2	1	0
物理页的物理地址 31~12	AVL	G	PAT	D	A	PCD	PWT	US	RW	P		

```

void MemoryManager::openPageMechanism()
{
    // 页目录表指针
    int *directory = (int *)PAGE_DIRECTORY;
    // 线性地址0~4MB对应的页表，一个页表 = 1024 个页表项 → 每个项管 4KB → 总共管 4MB 地址空间
    int *page = (int *) (PAGE_DIRECTORY + PAGE_SIZE);

    // 初始化页目录表
    memset(directory, 0, PAGE_SIZE);
    // 初始化线性地址0~4MB对应的页表
    memset(page, 0, PAGE_SIZE);

    int address = 0;
    // 将线性地址0~1MB恒等映射到物理地址0~1MB，为了访问方便
    // 内核启动初期必须这样，才能正常访问显存、BIOS 数据等
    for (int i = 0; i < 256; ++i)
    {
        // U/S = 1, R/W = 1, P = 1，物理地址 + 权限位 拼成一个页表项
        page[i] = address | 0x7;
        // 下一页
        address += PAGE_SIZE;
    }

    // 初始化页目录项

    // 0~1MB
    directory[0] = ((int)page) | 0x07;
    // 3GB的内核空间
    directory[768] = directory[0];
    // 最后一个页目录项指向页目录表，构造页目录项和页表项的虚拟地址而服务
    directory[1023] = ((int)directory) | 0x7;

    // 初始化cr3, cr0, 开启分页机制
    asm_init_page_reg(directory);

    printf("open page mechanism\n");
}

```

②将页目录表的地址写入 CR3

cr3（页目录基址寄存器 PDBR）寄存器保存的是页目录表的地址，使得 CPU 的 MMU 能够找到页目录表的地址，然后自动地将线性地址转换成物理地址

③将 CR0 的 PG 位置 1

启动分页机制开关是将控制寄存器 cr0 的 PG 位置 1，PG 位是 cr0 寄存器的第 31 位

```

; void asm_init_page_reg(int *directory);
asm_init_page_reg:
    push ebp
    mov ebp, esp

    push eax

    mov eax, [ebp + 4 * 2]
    mov cr3, eax; 放入页目录表地址
    mov eax, cr0
    or eax, 0x80000000
    mov cr0, eax    ; 置PG=1, 开启分页机制

    pop eax
    pop ebp

    ret

```

④开启分页机制

```
extern "C" void setup_kernel()
{
    // 中断管理器
    interruptManager.initialize();
    interruptManager.enableTimeInterrupt();
    interruptManager.setTimeInterrupt((void *)asm_time_interrupt_handler);

    // 输出管理器
    stdio.initialize();

    // 进程/线程管理器
    programManager.initialize();

    // 内存管理器
    memoryManager.openPageMechanism();
    memoryManager.initialize();
}
```

⑤编写 FIRST_THREAD 函数进行验证

```
printf("--- Paging & Resource Management ---\n");

// 从内核物理地址池申请 1 个物理页
uint32 paddr = memoryManager.allocatePhysicalPages(AddressPoolType::KERNEL, 1);
printf("Action: Allocated Physical Page at 0x%x\n", paddr);

// 偏移量 0x147A 是学号 5242 的十六进制
uint32 offset = 0x147A;

// 虚拟地址 A: 0x00000000 + offset (利用恒等映射)
// 虚拟地址 B: 0xc0000000 + offset (利用 directory[768] 的影子映射)
uint32 *vaddr_low = (uint32 *) (0x00000000 + offset);
uint32 *vaddr_high = (uint32 *) (0xc0000000 + offset);

uint32 magic_data = 0x5242ABCD;

*vaddr_low = magic_data;
printf("Step 1: Write 0x%x to [Virtual] 0x%x\n", magic_data, (uint32)vaddr_low);

// 从高端虚拟地址读取, 证明二级页表项 PDE[768] 指向正确
uint32 read_data = *vaddr_high;
printf("Step 2: Read from [High-Virtual] 0x%x, value = 0x%x\n", (uint32)vaddr_high, read_data);

if (read_data == magic_data) {
    printf("Result: SUCCESS! Paging mapping is consistent.\n");
}

// 释放申请的物理页
memoryManager.releasePhysicalPages(AddressPoolType::KERNEL, paddr, 1);
printf("Action: Released Physical Page at 0x%x\n", paddr);

uint32 paddr_new = memoryManager.allocatePhysicalPages(AddressPoolType::KERNEL, 1);
printf("Verification: Re-allocated Physical Page at 0x%x\n", paddr_new);

if (paddr == paddr_new) {
    printf("Final Result: Resource Management (Alloc/Release) is CORRECT.\n");
}
```

● 实验结果展示:

①

A. 资源申请与释放

- 分配: Action: Allocated Physical Page at 0x200000
- 释放与重分配:
 - 在测试中先执行了 Released Physical Page at 0x200000。
 - 随后立即进行再次分配, 结果显示 Verification: Re-allocated Physical Page at 0x200000。

B. 虚拟地址空间管理与过程

使用学号后四位 5242（十六进制为 0x147A）作为页内偏移。

- 虚拟地址信息：
 - 低端虚拟地址 0x147A：由 directory[0] 管理。在恒等映射下，它代表虚拟空间最底部的地址。
 - 高端虚拟地址 0xC000147A：由 directory[768] 管理。
 - **过程解释：** Step 1 向 0x147A 写入特征值 0x5242ABCD，Step 2 尝试从 0xC000147A 读取。结果显示 value = 0x5242ABCD，证明了两个不同的虚拟地址通过二级页表准确指向了同一个物理存储单元。
- 物理地址信息：0x147A

结论：

1. 映射机制有效：二级页表正确建立，MMU 能够根据 cr3 指向的页目录表完成地址翻译。
2. 空间隔离与联动：成功实现了虚拟地址空间（3GB 内核空间）对物理内存的管理。
3. 资源循环利用：物理池的分配与释放函数在分页开启后依然能稳定满足上层需求。

```
件 Booting from Hard Disk...
open page mechanism
total memory: 133038080 bytes ( 126 MB )
kernel pool
  start address: 0x200000
  total pages: 15984 ( 62 MB )
  bitmap start address: 0x10000
user pool
  start address: 0x4070000
  total pages: 15984 ( 62 MB )
  bit map start address: 0x107CE
--- Paging & Resource Management ---
Action: Allocated Physical Page at 0x200000
Step 1: Write 0x5242ABCD to [Virtual] 0x147A
Step 2: Read from [High-Virtual] 0xC000147A, value = 0x5242ABCD
Result: SUCCESS! Paging mapping is consistent.
Action: Released Physical Page at 0x200000
Verification: Re-allocated Physical Page at 0x200000
Final Result: Resource Management (Alloc/Release) is CORRECT.
```

②相比于一级页表，使用二级页表会带来的优势：

1. **离散化存储：**页表本身不再需要 4MB 的连续大块物理空间。

2. **按需创建**：只有当某个虚拟地址段被访问时，才需要创建对应的二级页表，极大节省了进程初始化时的内存开销。

3. **多映射共享**：通过在页目录中填入相同的页表地址（如第 0 项和第 768 项），实现了用同一套物理内存支撑不同虚拟地址段的精妙布局。

----- 实验任务 3 -----

- **任务要求**：虚拟页内存管理的实现——复现“虚拟页内存管理”代码
- 结合代码，描述虚拟页内存分配的三个基本步骤，以及虚拟页内存的释放的过程。
- 构造测试用例来分析虚拟页内存管理的实现是否存在 bug，如果存在，则尝试修复并再次测试。否则，结合测试用例简要分析虚拟页内存管理的实现的正确性。
- 在 PDE (页目录项) 和 PTE (页表项) 的虚拟地址构造中，我们使用了第 1023 个页目录项。第 1023 个页目录项指向了页目录表本身，从而使得我们可以构造出 PDE 和 PTE 的虚拟地址。现在，我们将这个指向页目录表本身的页目录项放入第 1000 个页目录项，而不再是放入了第 1023 个页目录项。请借助第 1000 个页目录项，构造出第 141 个页目录项的虚拟地址，和第 891 个页目录项指向的页表中第 109 个页表项的虚拟地址。

● 思路分析：

1. 内存管理器初始化逻辑

- **内核虚拟空间提升**：通过设置 `KERNEL_VIRTUAL_START` 为 `0xc0100000`，将内核映射到虚拟地址的高 3GB 以上区域。这种设计确保了所有用户进程在 3GB-4GB 区域共享同一份内核映射。

2. 页内存分配的三个关键步骤

第一步：虚拟页分配 (`allocateVirtualPages`)

- 返回连续虚拟地址的起始点，确保逻辑上的连续性。
- 注意回收，防止内存泄漏

第二步：物理页分配（allocatePhysicalPages）

- 注意：物理页在内存中可以是离散的。

第三步：建立映射（connectPhysicalVirtualPage）

- 构造 PDE/PTE 虚拟地址：**利用页目录表最后一项指向自身的技巧，计算出用于访问页目录项和页表项的特定虚拟地址。
- 按需创建页表：**检查 PDE 是否存在。若不存在，则动态申请一个物理页作为新页表，并初始化。
- 填充 PTE：**将物理页地址和权限属性（如 0x7）填入对应的 PTE 中，完成绑定。

3. 内存释放与防泄漏机制（releasePages）

- 物理回收：**通过 vaddr2paddr 查找 PTE 中的物理基址，将其归还物理池
 - 断开映射：**务必将对应的 PTE 项置 0，防止已释放的地址被非法访问（野指针保护）
 - 虚拟回收：**将虚拟位图对应的位置为可用
 - 错误处理：**若分配过程中某一步失败，必须回溯释放已分配的所有部分，确保系统内存不会因分配中断而导致永久损耗
- 实验步骤：

初始化

①在 MemoryManager 的基础上加入内核虚拟地址池：

```
class MemoryManager
{
public:
    // 可管理的内存容量
    int totalMemory;
    // 内核物理地址池
    AddressPool kernelPhysical;
    // 用户物理地址池
    AddressPool userPhysical;
    // 内核虚拟地址池
    AddressPool kernelVirtual;
```

②修改内存管理的初始化函数方法：

▲在 32 位系统中，虚拟地址空间总量为 4GB。操作系统通常将其划分为：

- **用户空间 (0~3GB)**：每个进程独有，互相隔离。
- **内核空间 (3GB~4GB)**：所有进程共享。

▲二级页表结构中，一个页目录项 (PDE) 管理 4MB 的空间。

- $3\text{GB} / 4\text{MB} = 768$ 。因此，从第 768 个 PDE 开始，直到第 1023 个，正好覆盖了 $768 \times 4\text{MB} = 3\text{GB}$ 到 $1024 \times 4\text{MB} = 4\text{GB}$ 的范围。
- **核心逻辑**：为了让所有用户进程看到的内核都是一样的
`directory[768] = directory[0];`
操作系统要求所有进程的页目录表中，第 768~1023 项的内容必须指向相同的物理页表。
- **地址等价性**：address + 0xc0000000 指向的 PDE（位于 768 项之后）与 address 指向的 PDE（位于 0 项之后）内容一致，最终它们会进入同一个页表，指向同一个物理地址

▲预留空间：255 个页表加上 1 个页目录表（第 1023 个页目录项指向了页目录表本身）

- **作用**：通过特定的虚拟地址（如 0xffff000），内核可以像读写普通数据页一样修改当前的页目录表和页表，而无需关闭分页机制。

▲ **物理现实**：内核代码和 BIOS/显存通常存放在物理内存的低端（0~1MB）。

虚拟需求：为了安全和统一，我们希望内核在“高位”运行（3GB 以上）。

起始地址计算：

- 内核在低端占用了 1MB
- 内核可分配的虚拟内存池不能从 0xc0000000 开始（那里已经被映射给物理 0 字节了），而要从 $0xc0000000 + 0x100000 = 0xc0100000$ 开始。

结果：程序通过访问 0xc0100000，在二级页表的转换下，实际上访问的是物理内存 1MB 往后的空间

```

int freePages = freeMemory / PAGE_SIZE;
int kernelPages = freePages / 2;
int userPages = freePages - kernelPages;

int kernelPhysicalStartAddress = usedMemory;
int userPhysicalStartAddress = usedMemory + kernelPages * PAGE_SIZE;

int kernelPhysicalBitMapStart = BITMAP_START_ADDRESS;
int userPhysicalBitMapStart = kernelPhysicalBitMapStart + ceil(kernelPages, 8);
//安排内核虚拟地址空间的位图在用户物理地址空间的位图之后
int kernelVirtualBitMapStart = userPhysicalBitMapStart + ceil(userPages, 8);

kernelPhysical.initialize(
    (char *)kernelPhysicalBitMapStart,
    kernelPages,
    kernelPhysicalStartAddress);

userPhysical.initialize(
    (char *)userPhysicalBitMapStart,
    userPages,
    userPhysicalStartAddress);

kernelVirtual.initialize(
    (char *)kernelVirtualBitMapStart,
    kernelPages,
    //内核提升到3GB~4GB的虚拟地址空间，让所有进程都能够共享内核分配的页内存
    KERNEL_VIRTUAL_START);

```

虚拟页内存分配

```

int MemoryManager::allocatePages(enum AddressPoolType type, const int count)
{
    // 第一步：从虚拟地址池中分配若干虚拟页
    int virtualAddress = allocateVirtualPages(type, count);
    if (!virtualAddress)
    {
        return 0;
    }

    bool flag;
    int physicalPageAddress;
    int vaddress = virtualAddress;

    // 依次为每一个虚拟页指定物理页
    for (int i = 0; i < count; ++i, vaddress += PAGE_SIZE)
    {
        flag = false;
        // 第二步：从物理地址池中分配一个物理页
        physicalPageAddress = allocatePhysicalPages(type, 1);
        if (physicalPageAddress)
        {
            //printf("allocate physical page 0x%x\n", physicalPageAddress);

            // 第三步：为虚拟页建立页目录项和页表项，使虚拟页内的地址经过分页机制变换到物理页内。
            flag = connectPhysicalVirtualPage(vaddress, physicalPageAddress);
        }
        else
        {
            flag = false;
        }
    }
}

```

分配失败注意内存空间的释放

```

// 分配失败，释放前面已经分配的虚拟页和物理页表
if (!flag)
{
    // 前i个页表已经指定了物理页
    releasePages(type, virtualAddress, i);
    // 剩余的页表未指定物理页
    releaseVirtualPages(type, virtualAddress + i * PAGE_SIZE, count - i);
    return 0;
}

return virtualAddress;

```

①从虚拟地址池中分配若干连续的虚拟页——`allocateVirtualPages` 函数

```

int MemoryManager::allocateVirtualPages(enum AddressPoolType type, const int count)
{
    int start = -1;
    //没有实现用户进程，能够分配页内存的地址池只有内核虚拟地址池
    if (type == AddressPoolType::KERNEL)
    {
        start = kernelVirtual.allocate(count);
    }

    return (start == -1) ? 0 : start;
}

```

②对每一个虚拟页，从物理地址池中分配 1 页——调用 `allocatePhysicalPages` 即可

③为虚拟页建立页目录项和页表项，使虚拟页内的地址经过分页机制变换到物理页内——`connectPhysicalVirtualPage`

▲虚拟页到物理页的映射关系

- CPU先取虚拟地址的31-22位 $virtual[31 : 22]$ ，在cr3寄存器中找到页目录表的物理地址，然后根据页目录表的物理地址在页目录表中找到序号为 $virtual[31 : 22]$ 的页目录项，读取页目录项中的页表的物理地址。
- CPU再取虚拟地址的21-12位 $virtual[21 : 12]$ ，根据第一步取出的页表的物理地址，在页表中找到序号为 $virtual[21 : 12]$ 的页表项，读取页表项中的物理页的物理地址 $physical$ 。
- 用物理页的物理地址的31-12位 $physical[31 : 12]$ 替换虚拟地址的31-12位 $virtual[31 : 12]$ 得到的结果就是虚拟地址对应的物理地址。

构造出虚拟地址对应的页目录项和页表项虚拟地址：为了修改页目录项和页表项的物理地址(注意区别物理地址和虚拟地址)

前提条件：在开启分页机制的时候，我们设置最后一个页目录项指向了页目录表

```

// 最后一个页目录项指向页目录表
directory[1023] = ((int)directory) | 0x7;

```

a. 页内偏移（低 12 位）：找到具体的页目录项

$$pde[11 : 0] = 4 \times virtual[31 : 22]$$

b. 找到页表（中间 10 位）

$$pde[21 : 12] = 111111111_2$$

c. 页目录项的序号（高 10 位）

$$pde[31 : 22] = 1111111111_2$$

类似可构造页表项

$$pte[11 : 0] = 4 \times virtual[21 : 12]$$

$$pte[21 : 12] = virtual[31 : 22]$$

$$pte[31 : 22] = 111111111_2$$

综上有

```
int MemoryManager::toPDE(const int virtualAddress)
{
    return (0xffff000 + (((virtualAddress & 0xffc00000) >> 22) * 4));
}

int MemoryManager::toPTE(const int virtualAddress)
{
    return (0xffc00000 + ((virtualAddress & 0xffc00000) >> 10) + (((virtualAddress & 0x003ff000) >> 12) * 4));
}
```

物理页到虚拟页映射：

```
bool MemoryManager::connectPhysicalVirtualPage(const int virtualAddress, const int physicalPageAd
{
    // 计算虚拟地址对应的页目录项和页表项
    int *pde = (int *)toPDE(virtualAddress);
    int *pte = (int *)toPTE(virtualAddress);

    // 页目录项无对应的页表，先分配一个页表
    if(!(*pde & 0x00000001))
    {
        // 从内核物理地址空间中分配一个页表
        int page = allocatePhysicalPages(AddressPoolType::KERNEL, 1);
        if (page == -1)
        {
            return false;
        }
        // 使页目录项指向页表
        *pde = page | 0x7;
        // 初始化页表
        char *pagePtr = (char *)(((int)pte) & 0xffff000);
        memset(pagePtr, 0, PAGE_SIZE);
    }

    // 使页表项指向物理页
    *pte = physicalPageAddress | 0x7;

    return true;
}
```

虚拟页内存的释放：

```
void MemoryManager::releasePages(enum AddressPoolType type, const int virtualAddress, const int count)
{
    int vaddr = virtualAddress;
    int *pte;
    for (int i = 0; i < count; ++i, vaddr += PAGE_SIZE)
    {
        // 第一步，对每一个虚拟页，释放为其分配的物理页
        releasePhysicalPages(type, vaddr2paddr(vaddr), 1);

        // 设置页表项为不存在，防止释放后被再次使用
        pte = (int *)toPTE(vaddr);
        *pte = 0;
    }

    // 第二步，释放虚拟页
    releaseVirtualPages(type, virtualAddress, count);
}
```

①释放物理页：虚拟地址转换物理地址——访问页表项，取页表项内容的 31-12 位就是物理页的物理地址，最后替换虚拟地址的 31-12 位

```
int MemoryManager::vaddr2paddr(int vaddr)
{
    int *pte = (int *)toPTE(vaddr);
    int page = (*pte) & 0xffff000;
    int offset = vaddr & 0xfff;
    return (page + offset);
}
```

②释放虚拟页：调用地址池释放函数方法

```
void MemoryManager::releaseVirtualPages(enum AddressPoolType type, const int vaddr, const int count)
{
    if (type == AddressPoolType::KERNEL)
    {
        kernelVirtual.release(vaddr, count);
    }
}
```

测试用例构造

分别分配 100，10，100 的物理页的 3 个内存，然后释放 10 的物理页，最后创建一个 100 和一个 10 的物理页；

- 1， 查看虚拟页是否是连续的，二级页表下对应的物理页是否可以离散；
- 2， 查看释放后的虚拟页和物理页是否可被再利用的。

```

char *p1 = (char *)memoryManager.allocatePages(AddressPoolType::KERNEL, 100);
char *p2 = (char *)memoryManager.allocatePages(AddressPoolType::KERNEL, 10);
char *p3 = (char *)memoryManager.allocatePages(AddressPoolType::KERNEL, 100);

printf("%x %x %x\n", p1, p2, p3);

memoryManager.releasePages(AddressPoolType::KERNEL, (int)p2, 10);
p2 = (char *)memoryManager.allocatePages(AddressPoolType::KERNEL, 100);

printf("%x\n", p2);

p2 = (char *)memoryManager.allocatePages(AddressPoolType::KERNEL, 10);

printf("%x\n", p2);

```

使用第 1000 个页目录项指向页目录表本身

目标对象	PDX	PTX	Offset	十六进制结果
第 141 个 PDE	1000	1000	141×4 (0x234)	0xFA7E8234
第 891 个页表中的第 109 个 PTE	1000	891	109×4 (0x1B4)	0xFA37B1B4

● 实验结果展示：

1. 内存池初始化状态
2. 虚拟地址连续性验证

通过对 p1（100 页）、p2（10 页）、p3（100 页）的初始分配，可以观察到虚拟空间的连续性：

* 地址输出：p1=0xC0100000, p2=0xC0164000, p3=0xC016E000。

* 数学验证：

- p2 - p1 = 0x64000 字节。计算：100 页 $\times$ 4096 字节/页 = 409600 字节 = 0x64000。
- p3 - p2 = 0x0A000 字节。计算：10 页 $\times$ 4096 字节/页 = 40960 字节 = 0x0A000。

3. 内存回收与碎片填充验证

* 释放操作：releasePages(p2, 10)使得虚拟地址池在 0xC0164000 处产生了一个大小为 10 页的“空洞”。

* 大块申请：在 p3 之后分配了*0xC01D2000。

* 小块申请：0xC0164000 处的 10 页空闲块

```

Booting from Hard Disk...
open page mechanism
total memory: 133038080 bytes ( 126 MB )
kernel pool
    start address: 0x200000
    total pages: 15984 ( 62 MB )
    bitmap start address: 0x10000
user pool
    start address: 0x4070000
    total pages: 15984 ( 62 MB )
    bit map start address: 0x107CE
kernel virtual pool
    start address: 0xC0100000
    total pages: 15984 ( 62 MB )
    bit map start address: 0x10F9C
C0100000 C0164000 C016E000
C01D2000
C0164000

```

但是目前无法观察到物理地址分配的离散，所以修改 first_thread 函数

```

// 制造内存碎片：申请一系列单页
void* fragments[10];
for (int i = 0; i < 10; ++i) {
    fragments[i] = (void*)memoryManager.allocatePages(AddressPoolType::KERNEL, 1);
}

// 间隔释放：释放其中的偶数索引页
for (int i = 0; i < 10; i += 2) {
    memoryManager.releasePages(AddressPoolType::KERNEL, (int)fragments[i], 1);
}

// 申请一个多页内存块
// 此时分配器会优先寻找并填补刚才释放掉的、互不相邻的空洞
int count = 3;
char *p_test = (char *)memoryManager.allocatePages(AddressPoolType::KERNEL, count);

printf("--- Discrete Physical Address Verification ---\n");
printf("Virtual Start Address: %x\n", p_test);

for (int i = 0; i < count; ++i)
{
    uint32 vaddr = (uint32)p_test + i * PAGE_SIZE;
    uint32 paddr = memoryManager.vaddr2paddr(vaddr);
    printf("Virtual Page %d: 0x%x ==> Physical Page %d: 0x%x\n", i, vaddr, i, paddr);
}

```

得到结果

```

--- Discrete Physical Address Verification ---
Virtual Start Address: C010A000
Virtual Page 0: 0xC010A000 ==> Physical Page 0: 0x200000
Virtual Page 1: 0xC010B000 ==> Physical Page 1: 0x202000
Virtual Page 2: 0xC010C000 ==> Physical Page 2: 0x204000

```

通过人为制造内存碎片，成功观察到了线性连续的虚拟地址映射到非连续物理地址的现象（从 0x200000 跳跃到了 0x202000，中间越过了 0x201000 这一页。）

实验任务 4

● 任务要求：页面置换算法

实现一种理论课上学习到的虚拟内存管理中的页面置换算法,在虚拟页内存中实现页面的置换,比如下面所列算法的其中一种:

先进先出页面置换(FIFO).

最优页面置换(OPR).

最近最少使用页面置换(LRU)

最不经常使用页面置换(LFU)

● 思路分析：实现 FIFO 算法

利用虚拟地址在内核池中连续分配的特性,通过维护一个“全局起始指针”来模拟队列操作

1. 置换机制的设计逻辑

- **记录起始点:** 在 MemoryManager 中引入 virtual_pages 变量,其初始值为 0xC0100000(内核虚拟起始地址)。该变量充当 FIFO 队列的“队首”,指向当前内存中存在时间最久的虚拟页。
- **置换条件:** 当 allocateVirtualPages 无法在位图中找到连续的空闲页时(返回 0),说明虚拟或物理资源已耗尽。
- **滑动窗口回收:**
 - 通过 while 循环不断调用 releasePages(type, virtual_pages, 1) 回收最早分配的页面。
 - 每次回收后,将 virtual_pages 指针向后移动一个 PAGE_SIZE (4KB)。
 - **环形映射:** 当指针达到内核虚拟池末尾(约 0xC3F71000)时,重置回起始地址,实现地址池的循环利用。
- **重新尝试:** 直到回收了足够数量的页面,使 allocateVirtualPages 能够成功返回起始地址为止

2. 测试用例设计逻辑

1. 预填满：申请 15900 个页面（接近总数 15984），几乎耗尽所有内核内存资源。
2. 触发阈值：随后的 p2, p3, p4。由于剩余空间不足，此时系统必然触发 while 循环。

● 实验步骤：

①设置一个队列用于管理内存页面

```
class MemoryManager
{
public:
    // 可管理的内存容量
    int totalMemory;
    // 内核物理地址池
    AddressPool kernelPhysical;
    // 用户物理地址池
    AddressPool userPhysical;
    // 内核虚拟地址池
    AddressPool kernelVirtual;
    unsigned int virtual_pages;
```

在 initiate 里面初始化

```
this->virtual_pages = KERNEL_VIRTUAL_START;
```

②修改分配页面函数：实现页面替换 FIFO 策略

```
int MemoryManager::allocatePages(enum AddressPoolType type, const int count)
{
    if(count > 15984){
        printf("No enough pages in total!\n");
        return 0;
    }

    // 第一步：从虚拟地址池中分配若干虚拟页
    int virtualAddress = allocateVirtualPages(type, count);
    while(!virtualAddress){
        releasePages(type, virtual_pages, 1);
        virtual_pages += PAGE_SIZE;
        if(virtual_pages == (0x3e71000+0xC0100000)) //0x3E700是第15984页的末地址
            virtual_pages = KERNEL_VIRTUAL_START;
        virtualAddress = allocateVirtualPages(type, count);
    }
    printf("the first virtual page to exchange is 0x%x\n", virtual_pages);
}
```

③测试用例：

```

void first_thread(void *arg)
{
    //本实验中虚拟页可用总数为15984个，可以分配一个15900的虚拟页给进程再开启后续页面
    char* p1 = (char*)memoryManager.allocatePages(AddressPoolType::KERNEL, 15900);
    char* p2 = (char*)memoryManager.allocatePages(AddressPoolType::KERNEL, 100);
    char* p3 = (char*)memoryManager.allocatePages(AddressPoolType::KERNEL, 10);
    char* p4 = (char*)memoryManager.allocatePages(AddressPoolType::KERNEL, 10);

    printf("p1:%x p2:%x p3:%x p4:%x\n", p1, p2, p3, p4);

    asm_halt();
}

```

● 实验结果展示：

```

Booting from Hard Disk...
open page mechanism
total memory: 133038080 bytes ( 126 MB )
kernel pool
    start address: 0x200000
    total pages: 15984 ( 62 MB )
    bitmap start address: 0x10000
user pool
    start address: 0x4070000
reOffice Writer
    total pages: 15984 ( 62 MB )
    bit map start address: 0x107CE
kernel virtual pool
    start address: 0xC0100000
    total pages: 15984 ( 62 MB )
    bit map start address: 0x10F9C
the first virtual page to exchange is 0xC0100000
the first virtual page to exchange is 0xC0164000
the first virtual page to exchange is 0xC0164000
the first virtual page to exchange is 0xC0164000
p1:C0100000 p2:C0100000 p3:C3F1C000 p4:C3F26000

```

1. 现象观察

- **触发置换：**当连续申请 p1（15900 页，0x3E1C）和 p2（100 页，0x64）后，由于总页数超过了内核虚拟池的上限（15984 页），系统自动触发了置换逻辑。
- **指针滑动：**
 - 第一次置换发生的地址是 0xC0100000（内核虚拟池起始点）。
 - 随后的置换地址滑动到了 0xC0164000。这说明系统正在按分配的先后顺序，依次回收最早进入内存的页面以腾出空间。
- **地址获取：**
 - p1 和 p2 均获得了起始地址 0xC0100000
 - p3 和 p4 获得了位于地址池末尾的虚拟地址 0xC3F1C000 和

0xC3F26000

2. 逻辑验证分析

- **置换的有效性：**在 p2 申请 100 页时，由于剩余空间不足，系统通过 while 循环多次调用 releasePages 释放了旧页。最终 p2 打印出 0xC0100000，说明它成功“抢占”并重新利用了刚刚释放出来的、位于池首的虚拟地址空间。
- p3 和 p4 能够在高地址处成功分配

Section 5 实验总结与心得体会

1. 实验总结：本次实验主要包括了以下四个方面

- **物理页内存管理：**建立了 kernelPhysical 和 userPhysical 地址池，利用位图（Bitmap）管理物理页帧的分配与回收
- **二级分页机制实现：**通过配置页目录表（PDT）和页表（PT），并设置 CR3 寄存器开启分页。通过“递归映射”（将第 1023 个 PDE 指向 PDT 自身）解决了在虚拟地址空间中安全修改页表条目的难题
- **虚拟页内存管理：**引入了内核虚拟地址池 kernelVirtual。通过 allocatePages 函数将连续的虚拟地址映射到可能离散的物理地址上
- **页面置换算法（FIFO）：**针对物理资源有限的情况，通过维护 virtual_pages 指针实现了“先进先出”置换逻辑。当分配请求无法满足时，主动牺牲最早分配的页面（如 0xC0100000）

2. 实验心得体会

- ①分页机制是一种资源调度方法。专注于连续的虚拟空间，而内核则在幕后像利用零散物理页，节省了空间资源
- ②置换虽然解决了空间不足的问题，但频繁的 releasePages 和 PTE 修改会带来额外的计算开销，需要权衡好算法的利弊
- ③在实验过程中，变量初始化时机（如 virtual_pages 为 0 的错误）会导致严重的后果（如下）

```
bit map start address: 0x107C2
kernel virtual pool
start address: 0xC0100000
total pages: 15984 ( 62 MB )
bit map start address: 0x10F9C
the first virtual page to exchange is 0x0
the first virtual page to exchange is 0x0
the first virtual page to exchange is 0x0
the first virtual page to exchange is 0x0
C0100000 A 0 C3F30000
```

总的来说, 本次实验让我掌握了 PDE/PTE、CR3 以及地址池位图等技术细节, 让我理解了操作系统是如何在有限的硬件资源上通过逻辑设计实现高效的内存管理

Section 6 附录：参考资料清单

实验指导——[SYSU-2023-Spring-Operating-System: 中山大学 2023 学年春季操作系统课程 - Gitee.com](#)

实验参考——[OS lab7 实现虚拟内存的内存管理 - RogersLuo](#)

大模型——[操作系统内存管理基础实验 - Google Gemini](#)